Programming in Python
IDC 212
Lecture - 10

October 23, 2025

Contents

1. Questions

1.1 Chemistry Problem

1.2 Physics Problem

1.3 Image Compression

Questions

Radioactive Decay Chain: A $\rightarrow$ B $\rightarrow$ C

Scenario. A nucleus A decays to B with rate k_1, and B decays to C with rate k_2:

$$A \xrightarrow{k_1} B \xrightarrow{k_2} C$$

- A — parent (radioactive), B — intermediate (radioactive), C — final (stable).
- We assume first–order kinetics (rates $\propto$ current amount).
- Initial amounts: $A(0) = A_0$, $B(0) = 0$, $C(0) = 0$.

Closed-Form Solutions

With first–order kinetics, the concentrations are:

$$A(t) = A_0\, e^{-k_1 t}$$

$$B(t) = \frac{k_1 A_0}{k_2 - k_1}\left(e^{-k_1 t} - e^{-k_2 t}\right) \quad (k_1 \neq k_2)$$

$$C(t) = A_0 - A(t) - B(t) \quad \text{(conservation of total amount)}$$

Notes:

- A decays mono-exponentially.
- B rises then falls (formed from A, lost to C).
- C monotonically increases to A_0 as $t \to \infty$.

NumPy Implementation (1/2)

Goal: Compute $A(t)$, $B(t)$, $C(t)$ over a time grid.

```
import numpy as np  # arrays & exp

# Initial amounts (arbitrary units)
A0, B0, C0 = 100.0, 0.0, 0.0

# Rate constants (per time unit)
k1, k2 = 0.10, 0.05   # ensure k1 != k2

# Time vector: 0, 1, 2, ... , 49
t = np.arange(0, 50, 1, dtype=float)
```

NumPy Implementation (2/2)

```
# A(t): simple exponential decay
A = A0 * np.exp(-k1 * t)

# B(t): formed from A, lost to C
B = (k1 * A0 / (k2 - k1)) * (np.exp(-k1 * t) - np.exp(-k2 * t))

# C(t): whatever remains by conservation
C = A0 - A - B

# Print values at the final time step
print("Final concentrations at t =", t[-1])
print("A =", A[-1], "B =", B[-1], "C =", C[-1])

# Quick sanity check: total remains ~ A0
print("Total (A+B+C) =", (A[-1] + B[-1] + C[-1]))
```

What Students Should Notice

- **Vectorization:** $\exp$ is applied to the whole time array at once.
- **Conservation:** $A(t) + B(t) + C(t) \approx A_0$ (small numerical error is normal).
- **Role of rates:** Increasing k_1 speeds loss of A and earlier peak of B; increasing k_2 drains B faster into C.
- **Edge case:** If $k_1 = k_2$, then $B(t) = k_1 A_0 \, t \, e^{-k_1 t}$.

2D Mass–Spring System: Stiffness Matrix

We study a simple 2D mass–spring chain with the stiffness matrix

$$\boldsymbol{K} = \begin{bmatrix} 4 & -2 & 0 \\ -2 & 4 & -2 \\ 0 & -2 & 4 \end{bmatrix}.$$

- Symmetric and positive definite (SPD).
- Appears in finite difference/finite element models.
- We will factorize and interpret $\boldsymbol{K}$ with LU, QR, eigen, and SVD.

Why Different Factorizations

- **LU**: fast direct solve; with pivoting improves stability.
- **QR**: orthogonal $\boldsymbol{Q}$ is numerically stable ($\boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}$).
- **Eigen**: for symmetric $\boldsymbol{K}$, eigenvalues are real; here λ_i are proportional to squared natural frequencies, $\omega_i = \sqrt{\lambda_i}$.
- **SVD**: most stable decomposition.

Setup in NumPy and SciPy

```
import numpy as np
from numpy.linalg import eig, norm
from scipy.linalg import lu, qr, svd

K = np.array([[ 4.0, -2.0,  0.0],
              [-2.0,  4.0, -2.0],
              [ 0.0, -2.0,  4.0]], dtype=float)
```

LU Decomposition with Pivoting

SciPy returns $\boldsymbol{P}, \boldsymbol{L}, \boldsymbol{U}$ such that $\boldsymbol{P}\,\boldsymbol{K} = \boldsymbol{L}\,\boldsymbol{U}$.

```python
P, L, U = lu(K)                    # P K = L U
check_LU = np.allclose(P @ K, L @ U, atol=1e-10)
print("LU check (P@K == L@U):", check_LU)

# Solve example: K x = b via LU (using triangular solves would be ideal;
# here we just verify the factorization consistency)
b = np.array([1.0, 0.0, 0.0])
x_from_LU = np.linalg.solve(K, b) # reference solve
print("Solve reference x:", x_from_LU)
```

QR Decomposition and Orthogonality

$\boldsymbol{K} = \boldsymbol{Q}\,\boldsymbol{R}$ with $\boldsymbol{Q}^\top \boldsymbol{Q} = \boldsymbol{I}$.

```
Q, R = qr(K, mode="economic")     # K = Q R
orth_Q  = np.allclose(Q.T @ Q, np.eye(Q.shape[1]), atol=1e-10)
recon_Q = np.allclose(Q @ R, K, atol=1e-10)
print("Q^T Q == I:", orth_Q)
print("Q R == K:", recon_Q)
```

Eigen Decomposition and Natural Frequencies

For symmetric $\boldsymbol{K}$: $\boldsymbol{K} = \boldsymbol{V}\,\boldsymbol{\Lambda}\,\boldsymbol{V}^\top$ with orthonormal $\boldsymbol{V}$.

```python
evals, evecs = eig(K) # Eigenvalues/eigenvectors
# Force real (numerical noise on symmetric problems)
evals = np.real(evals); evecs = np.real(evecs)
idx = np.argsort(evals) # Sort by ascending eigenvalue
evals = evals[idx]
evecs = evecs[:, idx]
# Normalize eigenvectors and check orthogonality
for i in range(evecs.shape[1]):
    evecs[:, i] /= norm(evecs[:, i])
ortho = np.allclose(evecs.T @ evecs, np.eye(3), atol=1e-10)
# Natural frequencies (squared)
omegas = np.sqrt(evals)
print("eigenvalues:", evals)
print("orthonormal V:", ortho)
print("natural frequencies:", omegas)
```

SVD and Comparison to Eigenvalues

For SPD $\boldsymbol{K}$: singular values s_i equal eigenvalues λ_i.

```
U_svd, s, VT = svd(K, full_matrices=False)  # K = U diag(s) V^T
# Compare spectrum (sorted)
same_spec = np.allclose(np.sort(s), np.sort(evals), atol=1e-10)
print("singular values:", s)
print("SVD spectrum equals eigen spectrum:", same_spec)

# Reconstruction check
K_svd = (U_svd * s) @ VT
print("SVD reconstruction ok:", np.allclose(K, K_svd, atol=1e
    -10))
```

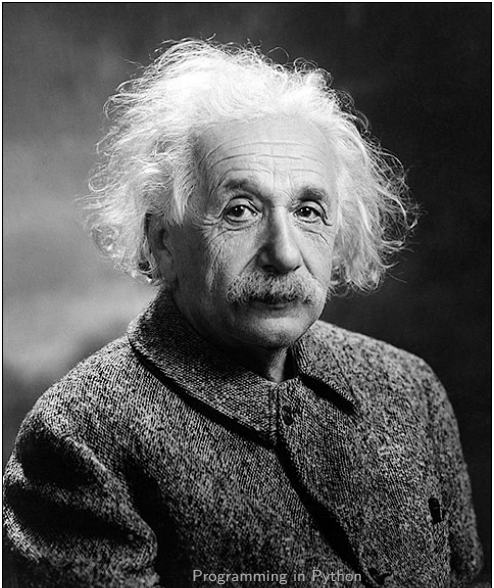
Compress a Grayscale Image with SVD

Goal. Use the top-k singular values to build a rank-k approximation of a 2-D image array.

Tasks:

- The original image and your compressed image for a fixed k.
- One sentence on how quality changed when you increased/decreased k.

Example image to use in class



Python: fixed-k SVD image compressor (save only)

```python
import numpy as np
from PIL import Image
# Step 1: Open image (force grayscale so A is 2-D)
img = Image.open("image1.jpg").convert("L")   # ensure grayscale
A = np.array(img, dtype=np.float32)           # float for SVD math
# Step 2: Apply SVD (reduced/"thin" form)
# full_matrices=False gives U:(m,k), S:(k,), VT:(k,n) with k=min(m,n)
U, S, VT = np.linalg.svd(A, full_matrices=False)

# Step 3: Keep only top-k singular values --------------------
k = 50                            # <= students can change this
k = int(min(k, min(A.shape)))     # keep k valid
Ak = (U[:, :k] * S[:k]) @ VT[:k, :]   # faster than U @ diag(S) @ VT

# Step 4: Convert back and save compressed image -------------
Ak_u8 = np.clip(Ak, 0, 255).astype(np.uint8)  # clip before casting
Image.fromarray(Ak_u8, mode="L").save("compressed_image.png")

print("Done. Wrote compressed_image.png")
```

What the code is doing (quick theory)

- **SVD:** $A = U\,\Sigma\,V^\top$, with singular values $\sigma_1 \geq \sigma_2 \geq \cdots$ on the diagonal of Σ.
- **Thin SVD (`full_matrices=False`):** keeps only the first $k = \min(m, n)$ columns/rows of $U, V^\top$ to save memory, which is sufficient for exact reconstruction when using all k.
- **Rank-k approximation:** By Eckart–Young, A_k is the best (least-squares) rank-k approximation.
- **Why clipping?** Numerical roundoff in A_k can push values slightly below 0 or above 255; `clip` avoids wrap-around when casting to `uint8`.

Tasks for students (quick checks)

1. Try $k \in \{1, 2, 10, 20, 50, 100\}$ and inspect the saved images.

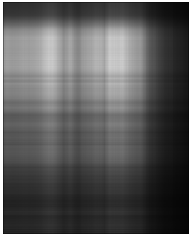
SVD Image Compression with Multiple k (Part 1)

```python
import numpy as np
from PIL import Image

# --- Load as grayscale and float ---
img = Image.open("image1.jpg").convert("L")
A = np.array(img, dtype=np.float32)

# --- Thin SVD ---
U, S, VT = np.linalg.svd(A, full_matrices=False)
m, n = A.shape
kmax = min(m, n)

# Choose the k's you want to try
k_list = [1, 2, 5, 10, 20, 50, 100]
k_list = [k for k in k_list if k <= kmax]
```

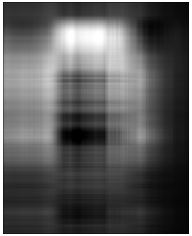
SVD Image Compression with Multiple k (Part 2)

```python
for k in k_list:
    # Rank-k reconstruction
    Ak = (U[:, :k] * S[:k]) @ VT[:k, :]

    # Save image (clip to [0,255])
    Ak_u8 = np.clip(Ak, 0, 255).astype(np.uint8)
    out_path = f"compressed_k{k}.png"
    Image.fromarray(Ak_u8, mode="L").save(out_path)

print("Done. Inspect the saved files to see quality vs k.")
```

Comparison of Compressed Images at Different k



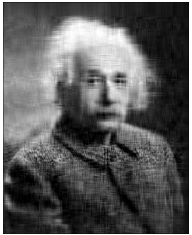
$k = 1$



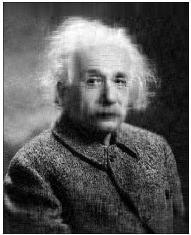
$k = 2$

$k = 5$

$k = 10$



$k = 20$



$k = 50$

Thank You